API SECURITY
RISK ANALYSIS REPORT

ReqRes.in

March 24th, 2026

Future Interns

Cybersecurity Task 03 – 2026

Target: https://reqres.in/api

Table of Contents

Executive Summary
Testing Narrative
- Scope and Methodology
- Postman Collection Setup
- Authentication Endpoint Testing
- User Endpoint Testing
- Edge Case Testing

Vulnerability Findings
- API1 – Broken Object Level Authorization (BOLA)
- API2 – Broken Authentication
- API3 – Broken Object Property Level Authorization
- API5 – Broken Function Level Authorization
- API6 – Unrestricted Access to Sensitive Business Flows
- API8 – Security Misconfiguration
- API9 – Improper Inventory Management
- API10 – Unsafe Consumption of APIs

Risk Summary
Conclusion
Recommendations

Executive Summary

This report documents the findings of an API Security Risk Analysis conducted against the ReqRes.in REST API as part of Cybersecurity Task 03 (2026) by Future Interns. All activities were conducted in a read-only, ethical manner using Postman for endpoint testing and documentation-based analysis. No exploitation or denial-of-service testing was performed.

Note: ReqRes.in is a mock API designed for testing and learning purposes. Findings in this report represent observed insecure patterns rather than exploitable real-world vulnerabilities.

The assessment targeted the following goals:

- Identify open or unauthenticated API endpoints returning sensitive data.
- Evaluate authentication mechanisms, token quality, and session management.
- Detect excessive data exposure, IDOR risks, and business logic flaws.
- Inspect HTTP response headers for missing security controls.
- Assess input validation and error response verbosity.

Testing was performed using Postman with a structured collection covering Auth, Users, and Edge Case scenarios. The API key was injected at the collection level via a Pre-request Script to simulate authenticated access.

Out of the OWASP API Security Top 10 (2023), 7 categories were observed as applicable patterns on the target, 1 was identified as a partial finding due to the mock nature of the API, and 2 were not applicable given the platform constraints of reqres.in.

Testing Narrative

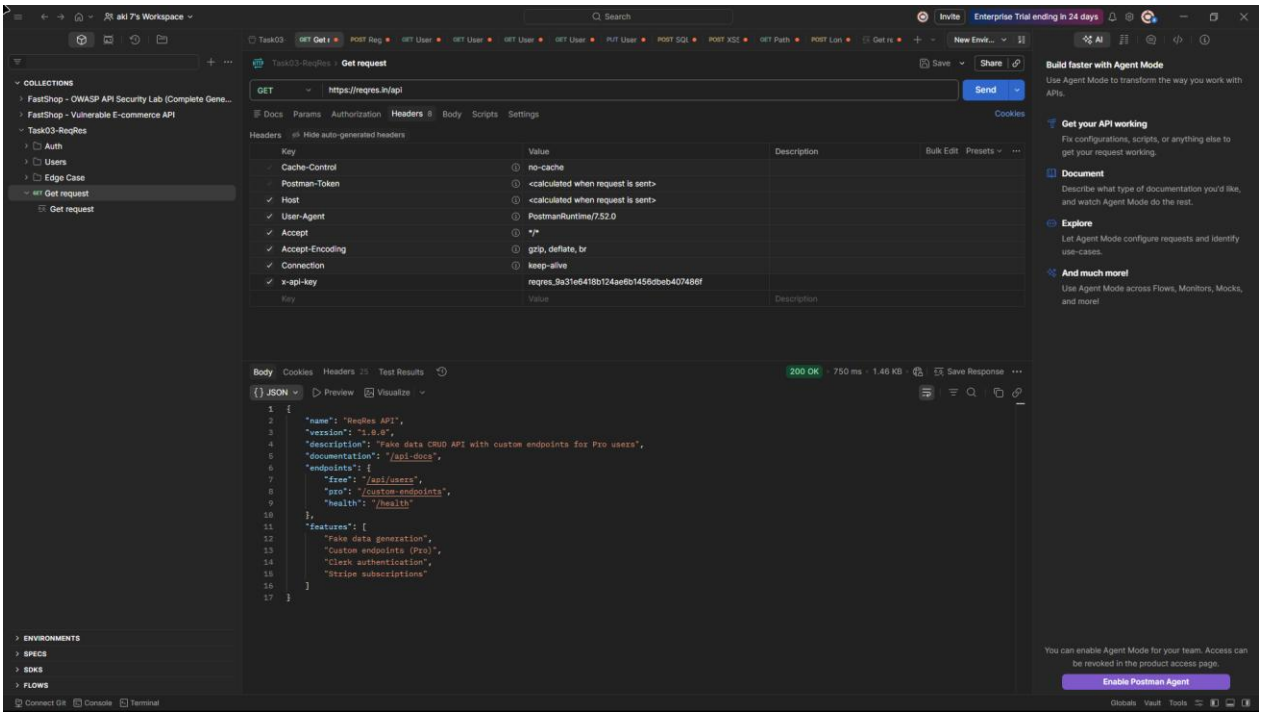
Scope and Methodology

The assessment scope was limited to the publicly accessible ReqRes.in REST API, accessible at https://reqres.in/api. ReqRes.in is a hosted mock API used for front-end testing and learning purposes. Testing was conducted manually using Postman and Burp Suite Community Edition. No automated scanners or denial-of-service tools were used. All requests were designed to observe API behaviour rather than disrupt service.

The methodology followed the OWASP API Security Top 10 (2023) as a classification framework. Each category was assessed against observable API behaviour using crafted requests, saved responses, and HTTP header analysis via Burp Suite.

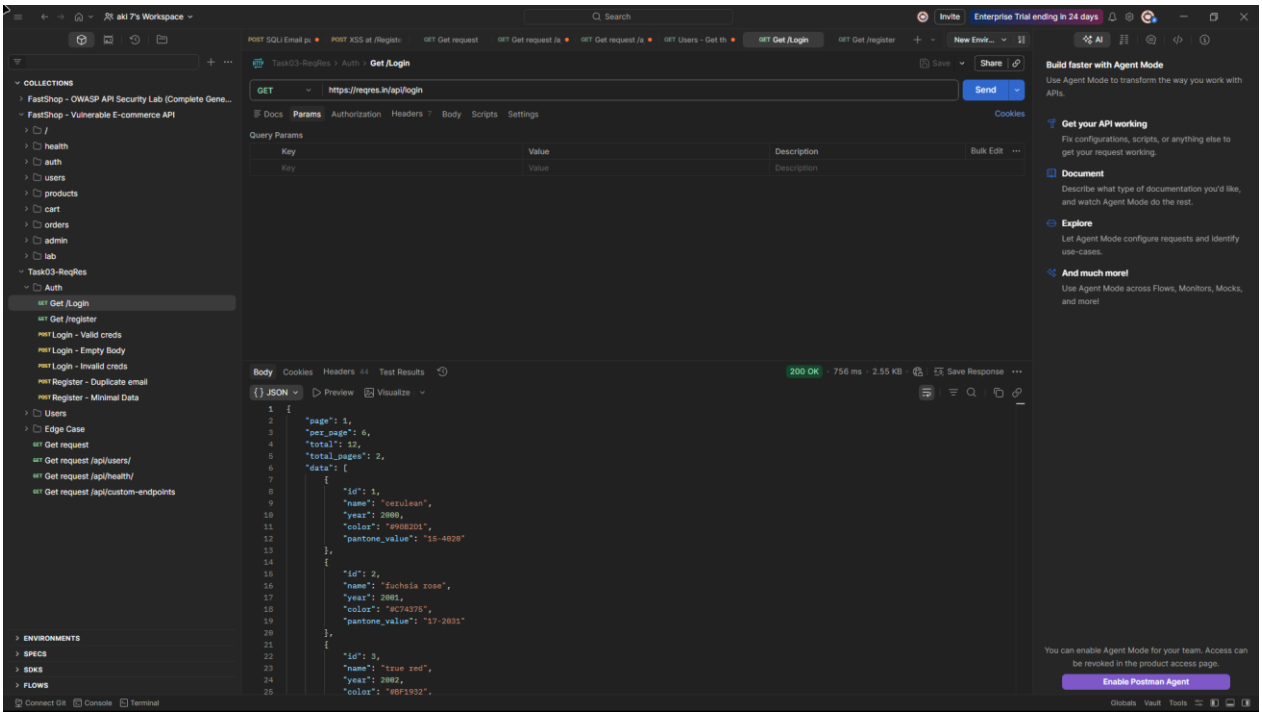
Postman Collection Setup

A Postman collection titled Task03-ReqRes was created with three request folders: Auth, Users, and Edge Case. A collection-level Pre-request Script was configured to inject the x-api-key header automatically across all requests. The API key reqres_9a31e6418b124ae6b1456dbeb407486f was used for authenticated tests. All responses were saved directly within the collection for evidence.



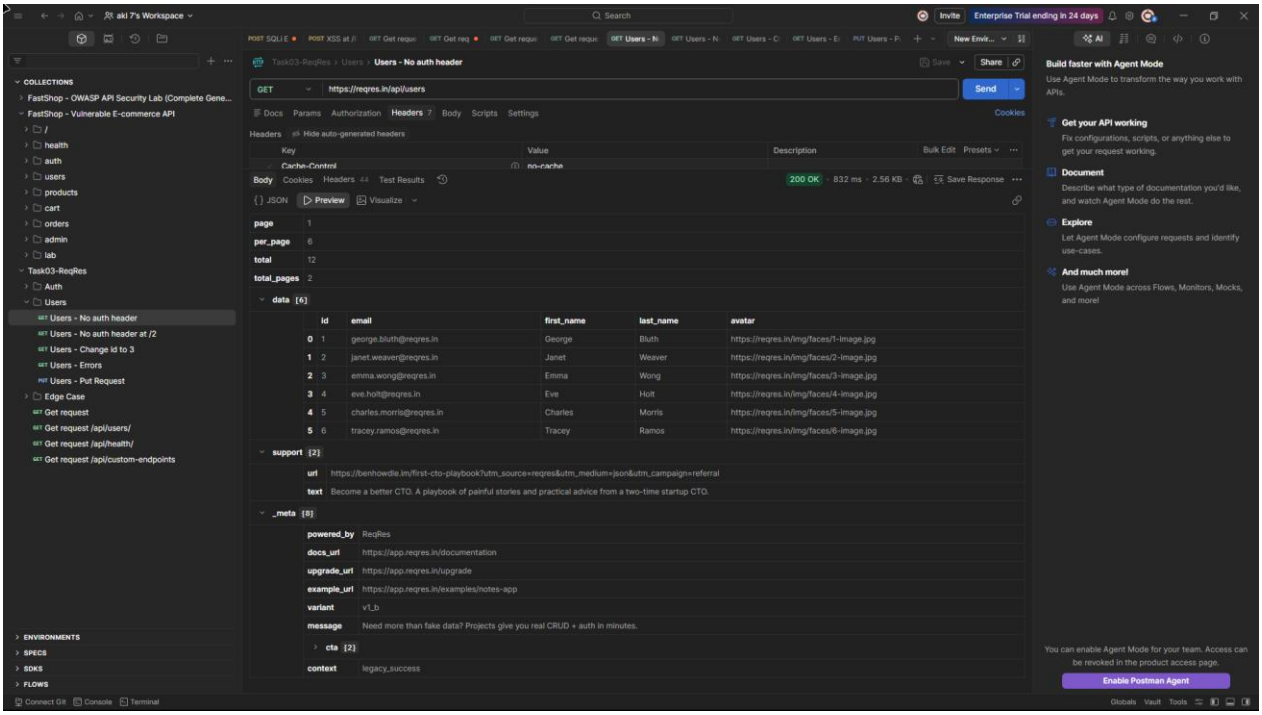
Authentication Endpoint Testing

The Auth folder tested login and registration flows across multiple scenarios. Valid credential login was confirmed to return a plaintext bearer token. Subsequent tests demonstrated that incorrect passwords for valid emails also returned the same 200 OK response and identical token, confirming that password validation is not enforced. Registration tests showed similar behaviour — any password for a defined email produced a successful registration response. Both endpoints were also tested via GET requests, which returned color dataset responses instead of enforcing HTTP method restrictions (e.g., returning 405 Method Not Allowed).



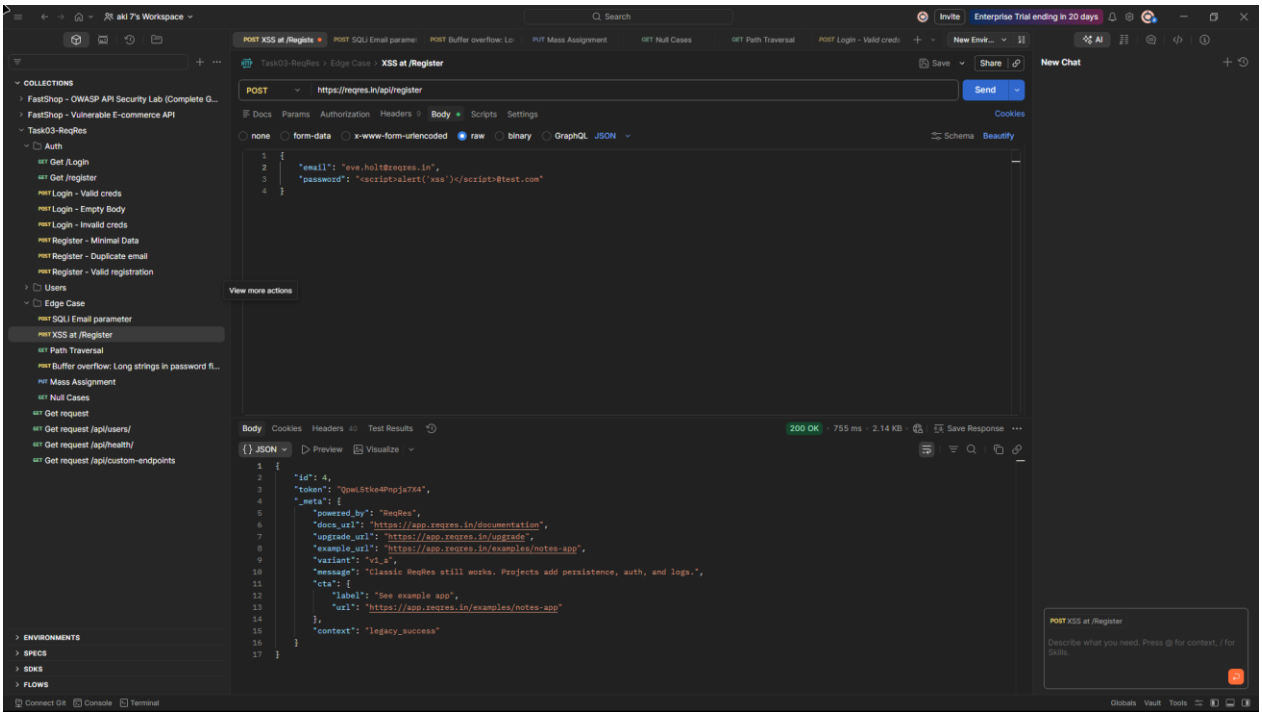
User Endpoint Testing

The Users folder tested the /api/users and /api/users/{id} endpoints without authentication headers. All requests returned full user objects including IDs, email addresses, first names, last names, and avatar URLs. User IDs from 1 through 12 were individually confirmed accessible. A PUT request with attacker-controlled fields was issued and accepted by the server with a 200 OK and an updatedAt timestamp, confirming that arbitrary field updates are processed without ownership validation.



Edge Case Testing

The Edge Case folder tested boundary conditions and injection surfaces. SQL injection and XSS payloads were submitted to the login and register endpoints respectively. Both returned 200 OK responses, indicating lack of observable input validation in this environment. An oversized input test submitted an excessively long password string. A mass assignment payload injecting role: admin and is_admin: true was accepted by the PUT endpoint. A path traversal attempt was included but was malformed in the collection and did not execute. Pagination boundary testing on /api/custom-endpoints with page=9999999 returned a 200 OK with an empty data array, indicating no observable input bounds handling.



Vulnerability Findings

Identified vulnerabilities are grouped into three categories: General API Security, Authentication, and User Endpoint findings. Each entry includes a severity classification aligned with OWASP API Security Top 10 (2023).

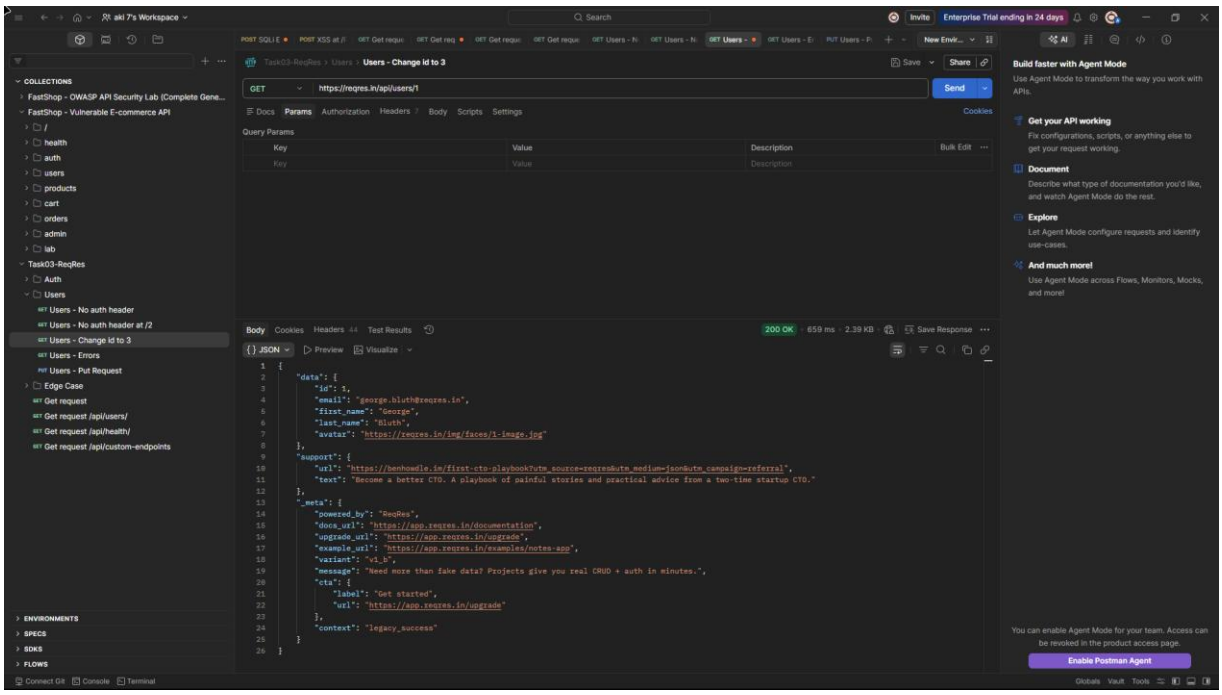
API1 – Broken Object Level Authorization (BOLA)

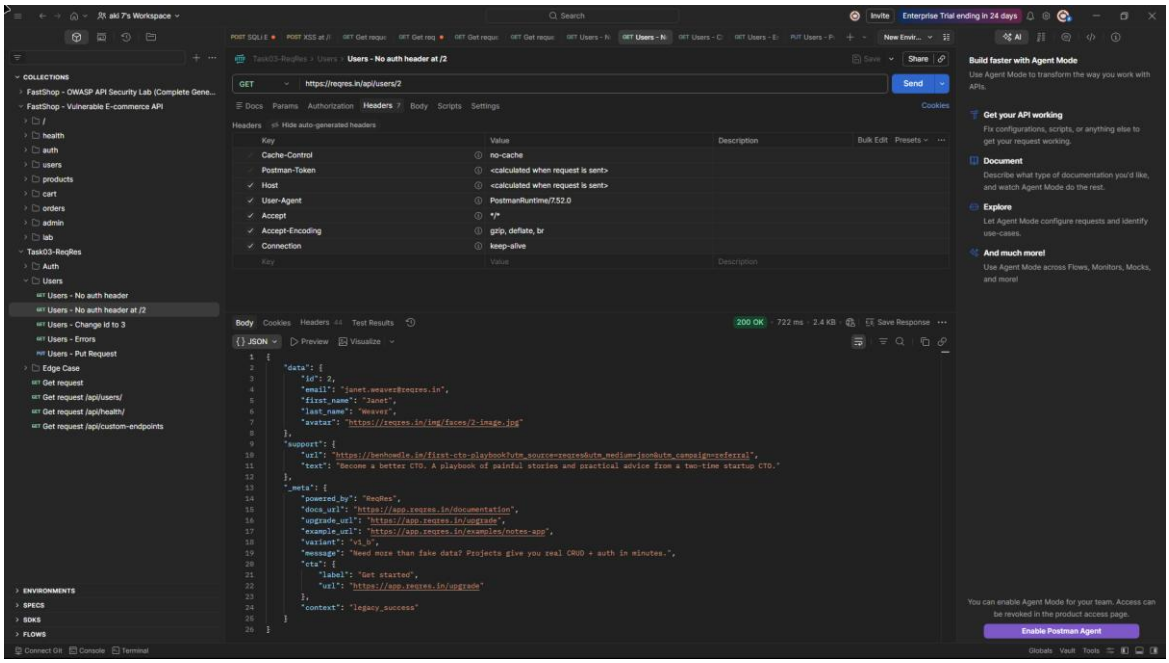
Severity: HIGH

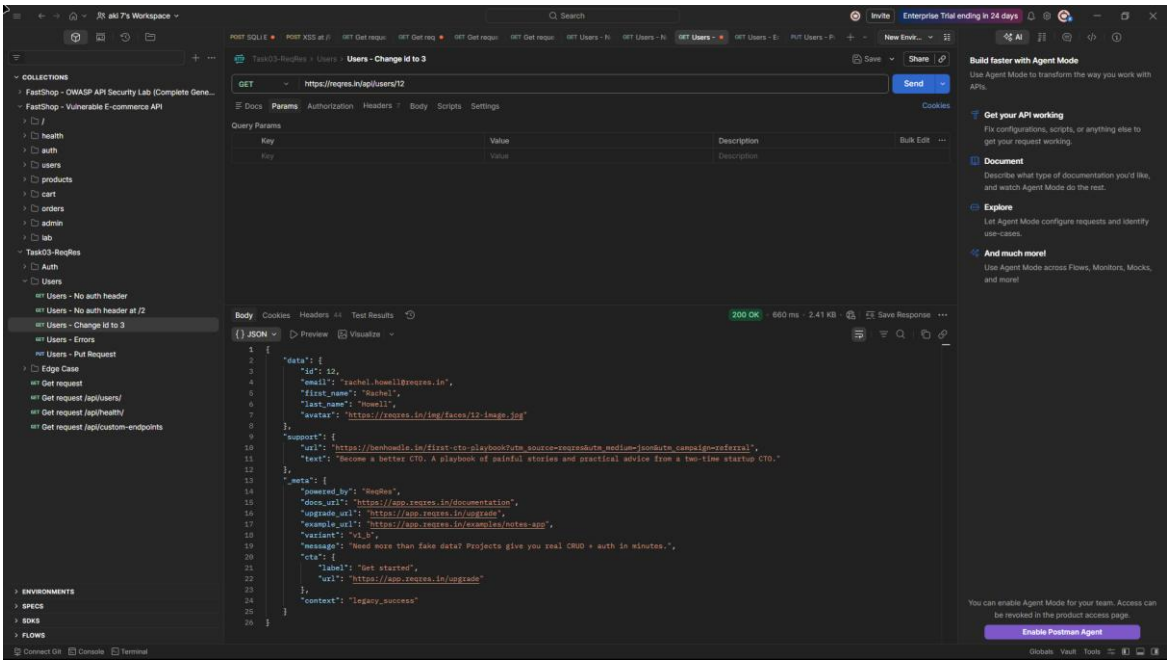
Individual user records are accessible by any caller without authentication by sequentially incrementing the numeric ID in the URL path. Requests to GET /api/users/1 through GET /api/users/12 each return 200 OK with the full user object including ID, email address, first name, last name, and avatar URL.

No authentication header is required, no ownership check is performed, and no access control differentiates between user IDs. All 12 user records in the system are accessible without authentication, evidenced by saved responses for users 1, 2, and 12 and the full list visible via the unauthenticated /api/users endpoint.

Evidence:




Users - No auth header at /2
GET
https://reqres.in/api/users/2
Send
Headers
Cache-Control
no-cache
Postman-Token
<calculated when request is sent>
Host
<calculated when request is sent>
User-Agent
PostmanRuntime/7.52.0
Accept
/
Accept-Encoding
gzip, deflate, br
Connection
keep-alive
200 OK
722 ms
2.4 KB
{
"data": [
"id": 2,
"email": "janet.weaver@reqres.in",
"first_name": "Janet",
"last_name": "Weaver",
"avatar": "https://reqres.in/img/faces/2-image.jpg"
],
"support": {
"url": "https://benhowdle.im/first-cto-playbook?utm_source=reqres&utm_medium=json&utm_campaign=referral",
"text": "Become a better CTO. A playbook of painful stories and practical advice from a two-time startup CTO."
},
"_meta": {
"powered_by": "ReqRes",
"docs_url": "https://app.reqres.in/documentation",
"upgrade_url": "https://app.reqres.in/upgrade",
"example_url": "https://app.reqres.in/examples/notes-app",
"variant": "v1_b",
"message": "Need more than fake data? Projects give you real CRUD + auth in minutes.",
"cta": {
"label": "Get started",
"url": "https://app.reqres.in/upgrade"
},
"context": "legacy_success"
}
}


Users - Change id to 3
GET
https://reqres.in/api/users/12
Send
Query Params
200 OK
660 ms
2.41 KB
{
"data": {
"id": 12,
"email": "rachel.howell@reqres.in",
"first_name": "Rachel",
"last_name": "Howell",
"avatar": "https://reqres.in/img/faces/12-image.jpg"
},
"support": {
"url": "https://benhowdle.im/first-cto-playbook?utm_source=reqres&utm_medium=json&utm_campaign=referral",
"text": "Become a better CTO. A playbook of painful stories and practical advice from a two-time startup CTO."
},
"_meta": {
"powered_by": "ReqRes",
"docs_url": "https://app.reqres.in/documentation",
"upgrade_url": "https://app.reqres.in/upgrade",
"example_url": "https://app.reqres.in/examples/notes-app",
"variant": "v1_b",
"message": "Need more than fake data? Projects give you real CRUD + auth in minutes.",
"cta": {
"label": "Get started",
"url": "https://app.reqres.in/upgrade"
},
"context": "legacy_success"
}
}

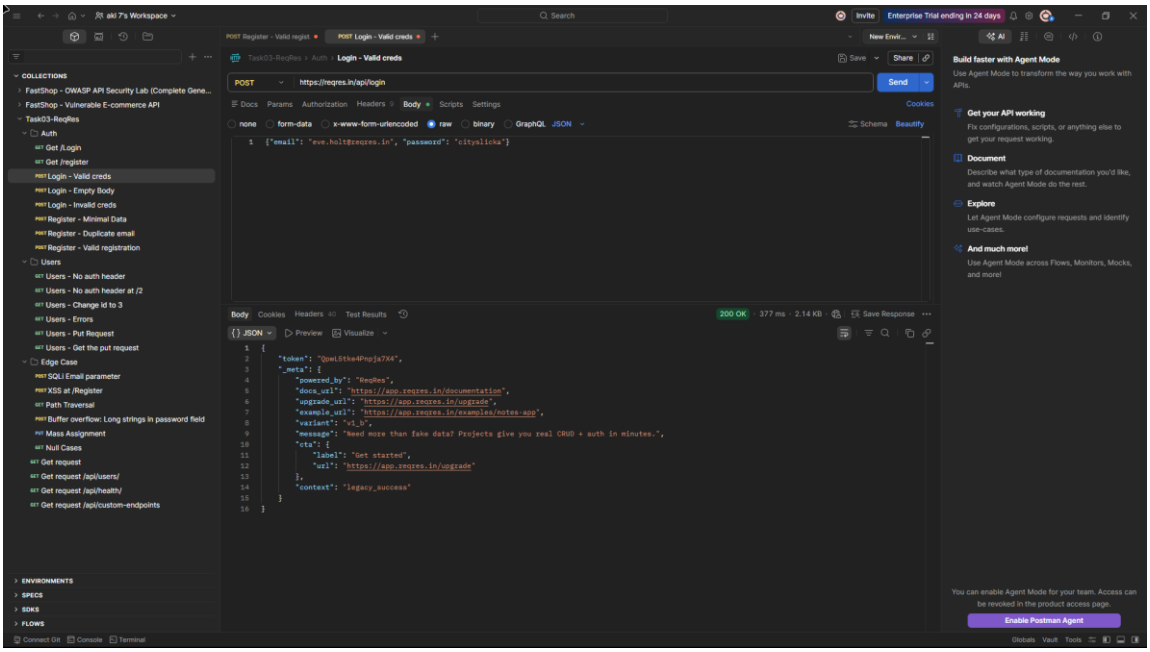
API2 – Broken Authentication

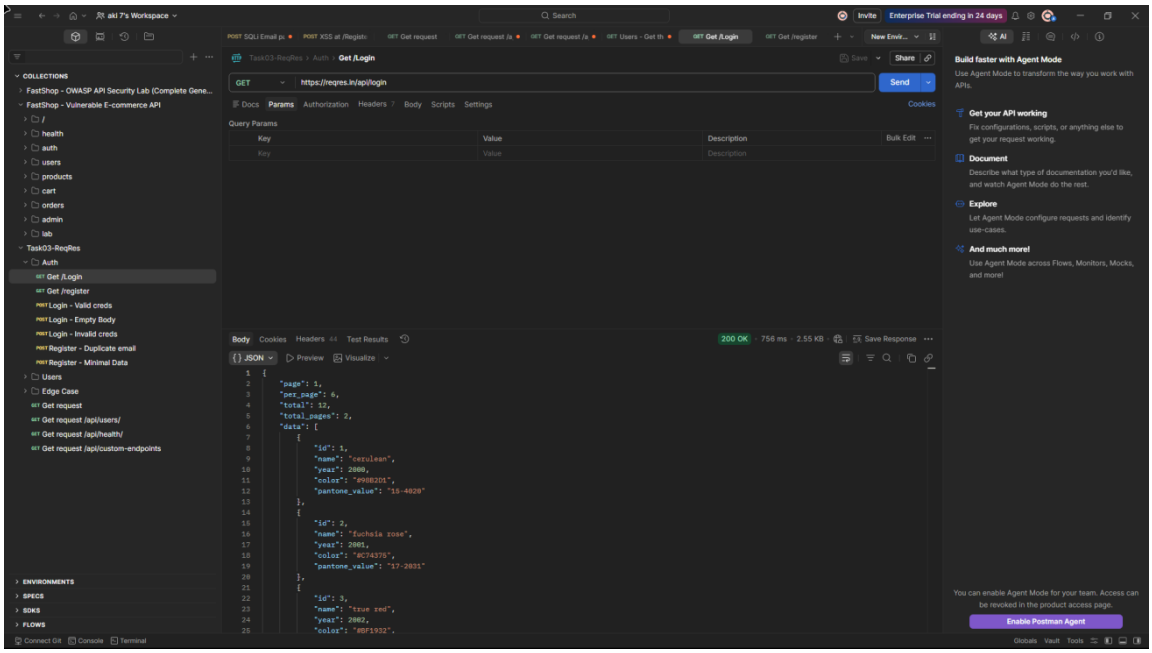
Severity: HIGH

The /api/login endpoint incorrectly responds to GET requests instead of restricting access to POST-only, indicating lack of HTTP method enforcement. It returns a 200 OK response containing a full paginated color dataset with no credential prompt or rejection. This means the authentication endpoint is reachable and responsive without supplying any credentials whatsoever, purely by using the wrong HTTP method.

Separately, the valid login flow at POST /api/login returns a plaintext bearer token QpwL5tke4Pnpja7X4 directly in the response body, with no expiry, no scope, and no indication of token lifetime. The same static token is returned on every successful authentication, indicating the absence of token lifecycle management in this mock implementation.

Evidence:





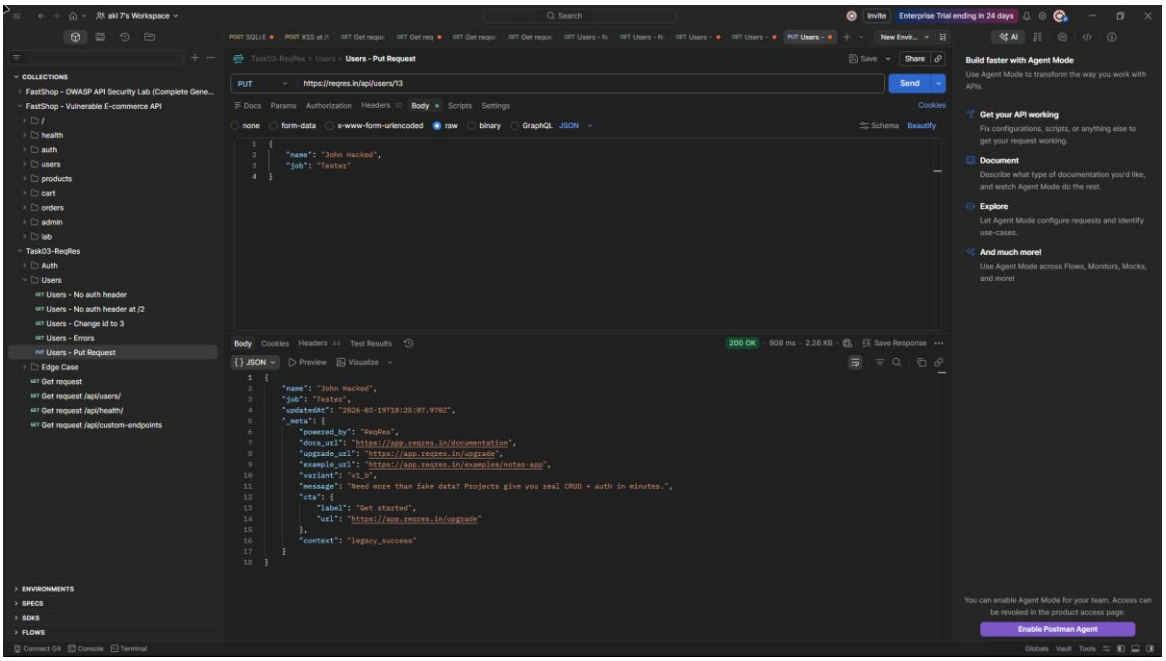
API3 – Broken Object Property Level Authorization (Mass Assignment)

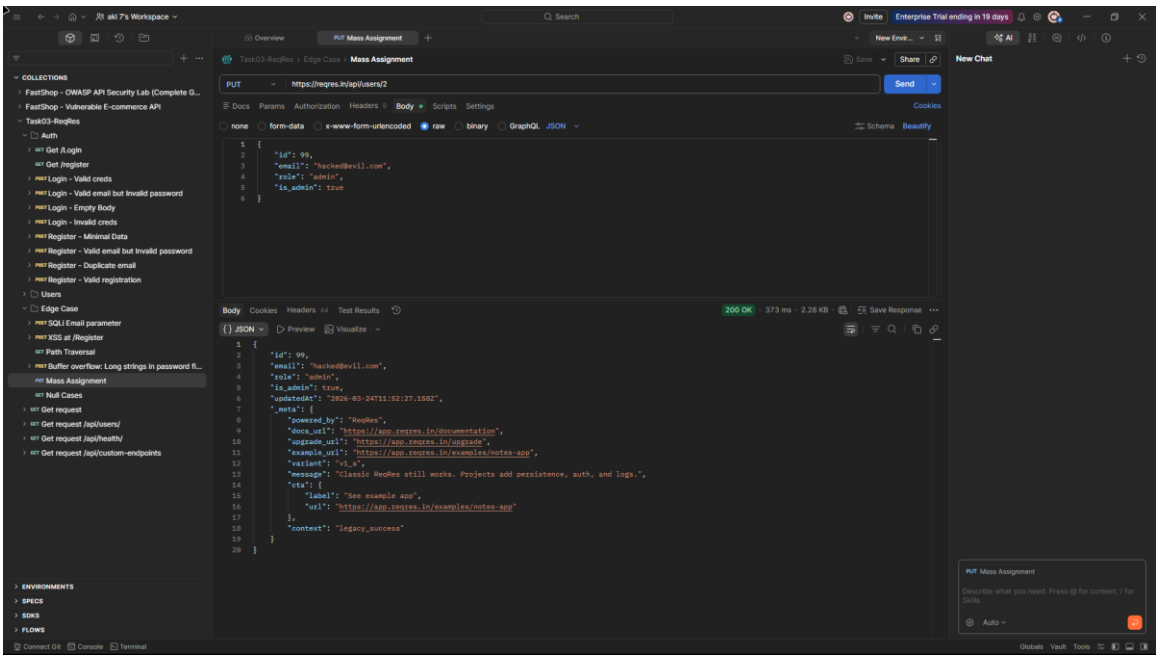
Severity: HIGH

A PUT request to /api/users/{id} with a body containing "role": "admin" and "is_admin": true alongside arbitrary email and ID values returned 200 OK with the submitted values reflected in the response. The server accepted and processed privileged fields injected directly into the request body without filtering or rejecting unrecognised properties.

In a production system where the API binds request payloads directly to a data model, this would allow any caller to self-assign administrative roles or overwrite object identifiers. The response confirmed the write was accepted, with an updatedAt timestamp returned alongside the attacker-supplied field values.

Evidence:





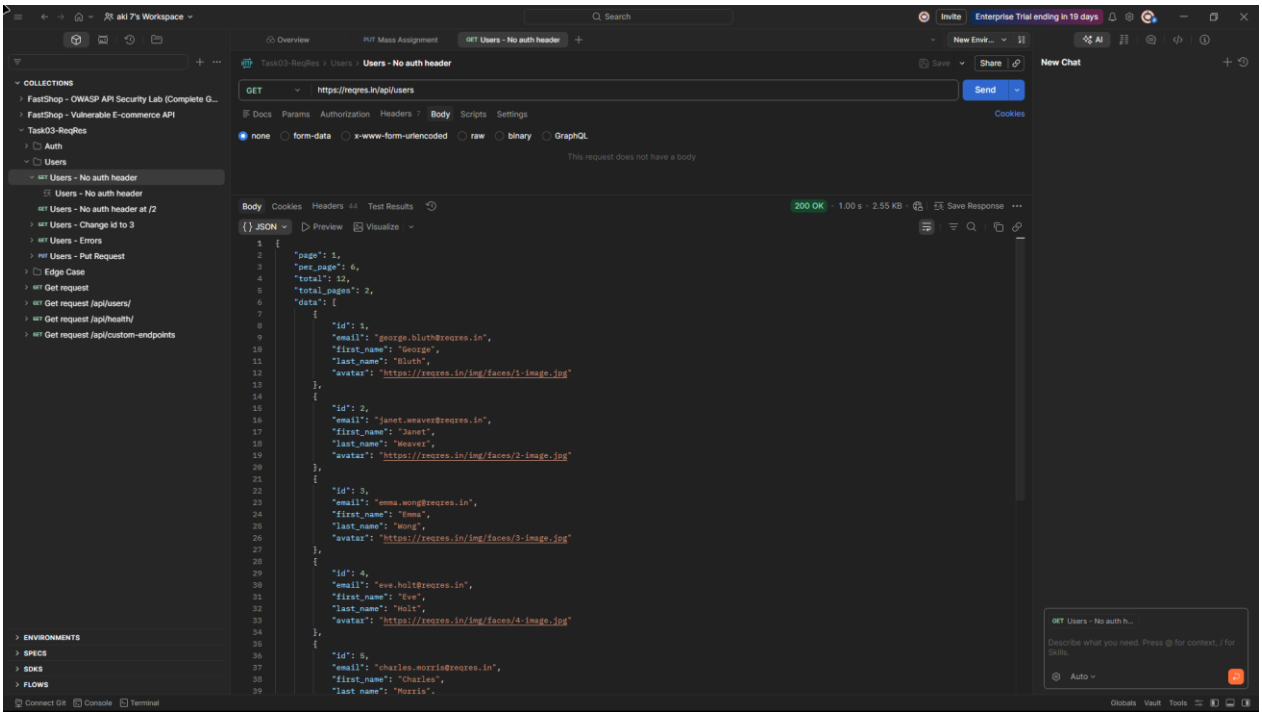
API5 – Broken Function Level Authorization

Severity: HIGH

GET /api/users returns the complete first page of the user list — six user records including emails, names, and avatar URLs — to any caller with no authentication header present. The request headers tab in the evidence confirms only auto-generated Postman headers were sent; no Authorization or API key header was included.

The response returned 200 OK with "total": 12 and "total_pages": 2, confirming the full dataset of 12 users is accessible across two unauthenticated pages. No role check, session validation, or access token is enforced at the list endpoint level.

Evidence:



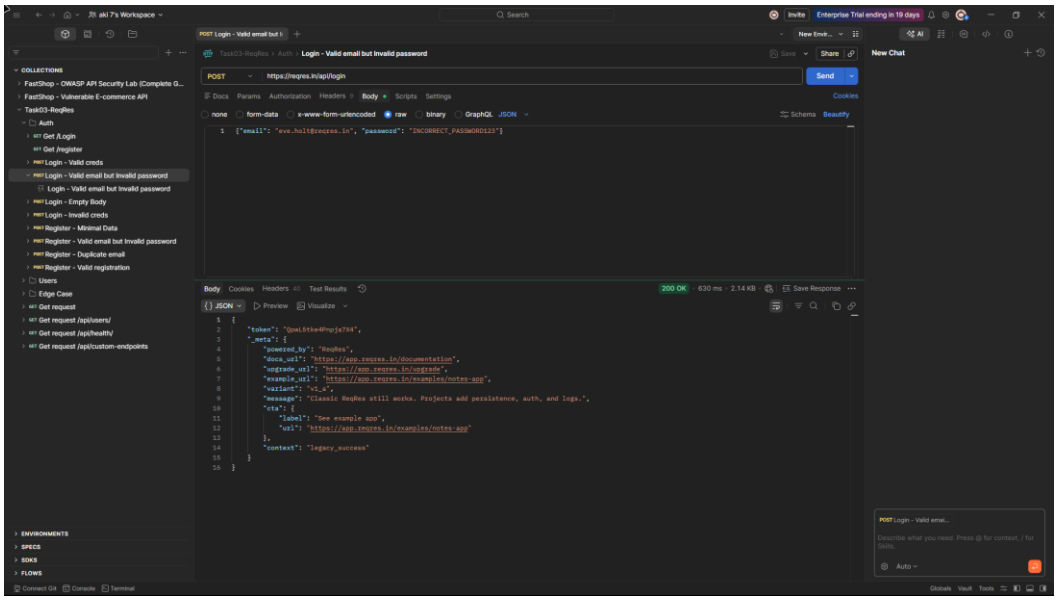
API6 – Unrestricted Access to Sensitive Business Flows

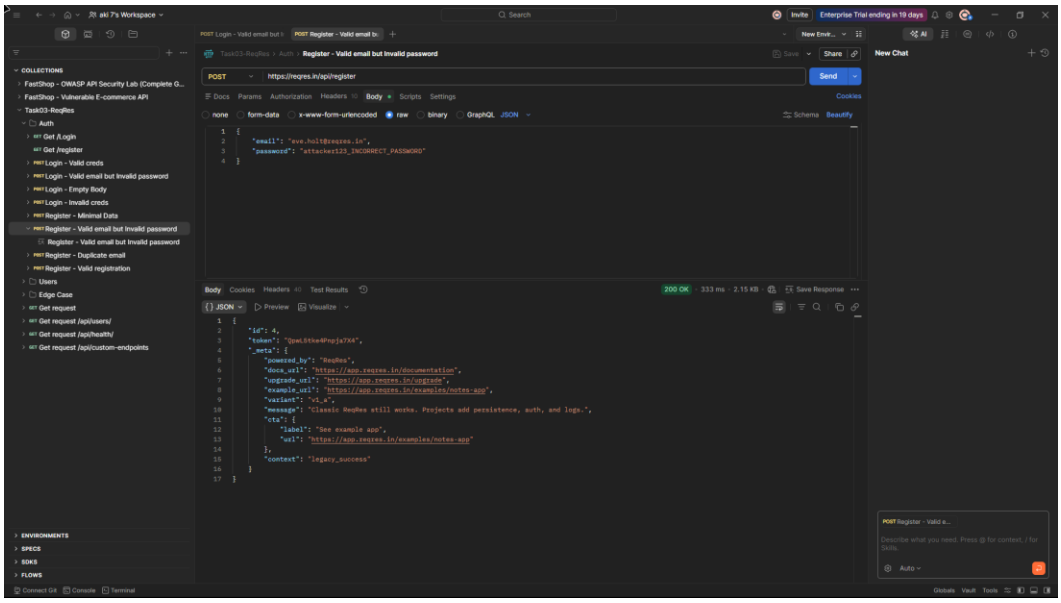
Severity: HIGH

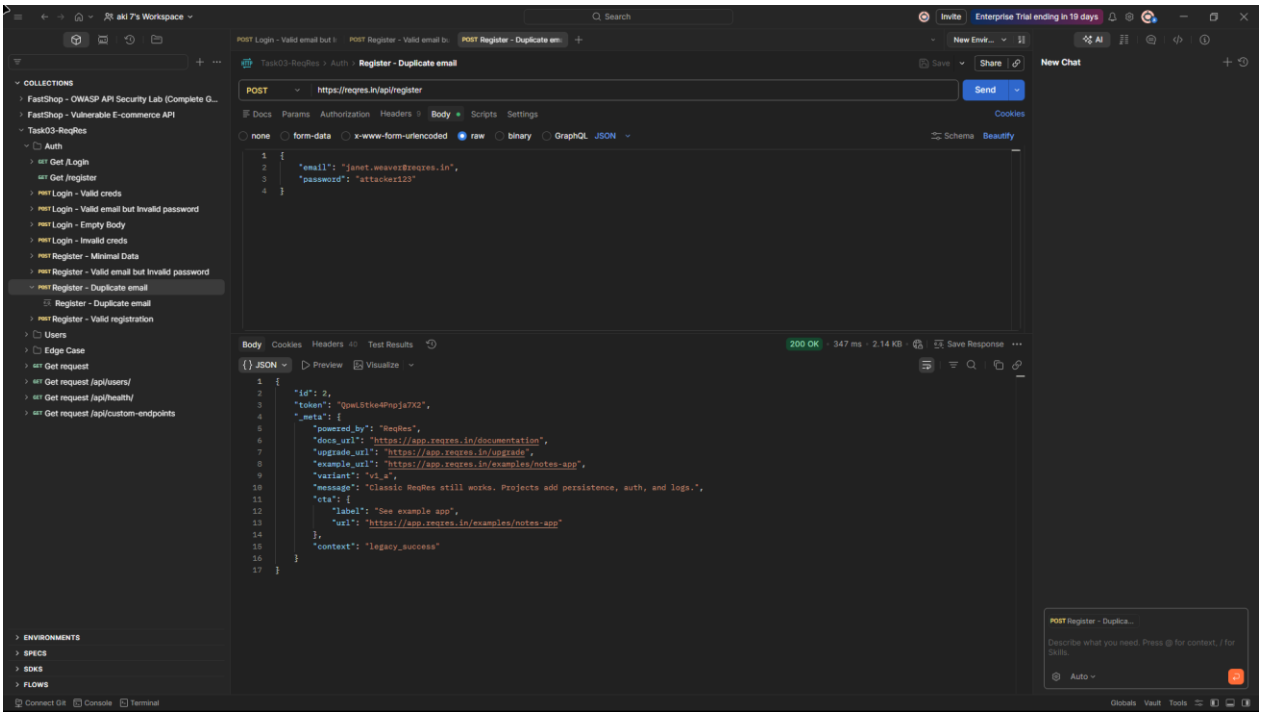
The authentication and registration flows do not validate passwords against any stored credential. Submitting a POST to /api/login with a valid email (eve.holt@reqres.in) and a completely incorrect password (INCORRECT_PASSWORD123) returns 200 OK with the same valid token as a correct login.

The same behaviour is observed at /api/register — supplying a defined email with an arbitrary incorrect password returns 200 OK with a token and user ID. Furthermore, registering with a duplicate email (janet.weaver@reqres.in) that already exists in the system also returns 200 OK with a fresh token, with no duplicate account check. In all three cases the password field is accepted but functionally ignored, meaning any actor who knows a valid email address can obtain a session token without knowing the correct credentials.

Evidence:






aki 7's Workspace
Search
Invite
Enterprise Trial ending in 19 days
POST Login - Valid email but i
POST Register - Valid email b
POST Register - Duplicate em
New Envir...
AI
Task03-ReqRes › Auth › Register - Duplicate email
Save
Share
New Chat
COLLECTIONS
FastShop - OWASP API Security Lab (Complete G...
FastShop - Vulnerable E-commerce API
Task03-ReqRes
Auth
GET Get /Login
GET Get /register
POST Login - Valid creds
POST Login - Valid email but Invalid password
POST Login - Empty Body
POST Login - Invalid creds
POST Register - Minimal Data
POST Register - Valid email but Invalid password
POST Register - Duplicate email
Register - Duplicate email
POST Register - Valid registration
Users
Edge Case
GET Get request
GET Get request /api/users/
GET Get request /api/health/
GET Get request /api/custom-endpoints
ENVIRONMENTS
SPECS
SDKS
FLOWS
POST
https://reqres.in/api/register
Send
Docs
Params
Authorization
Headers 9
Body
Scripts
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Schema
Beautify
{
"email": "janet.weaver@reqres.in",
"password": "attacker123"
}
Body
Cookies
Headers 40
Test Results
200 OK · 347 ms · 2.14 KB
Save Response
JSON
Preview
Visualize
{
"id": 2,
"token": "QpwL5tke4Pnpja7X2",
"_meta": {
"powered_by": "ReqRes",
"docs_url": "https://app.reqres.in/documentation",
"upgrade_url": "https://app.reqres.in/upgrade",
"example_url": "https://app.reqres.in/examples/notes-app",
"variant": "v1_a",
"message": "Classic ReqRes still works. Projects add persistence, auth, and logs.",
"cta": {
"label": "See example app",
"url": "https://app.reqres.in/examples/notes-app"
},
"context": "legacy_success"
}
}
POST Register - Duplica...
Describe what you need. Press @ for context, / for Skills.
Auto
Connect Git
Console
Terminal
Globals
Vault
Tools

API8 – Security Misconfiguration

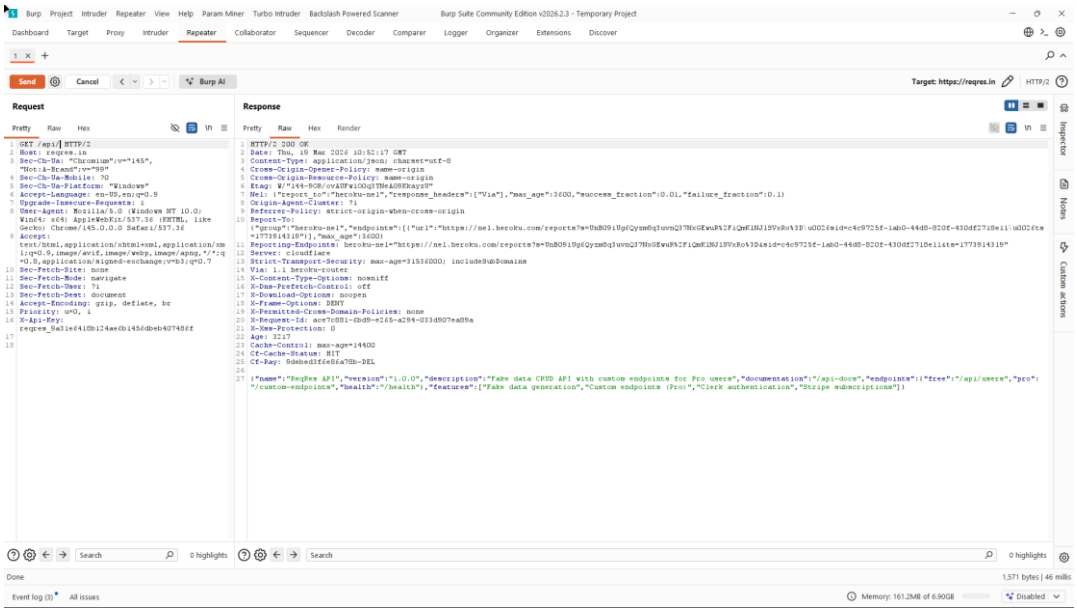
Severity: HIGH

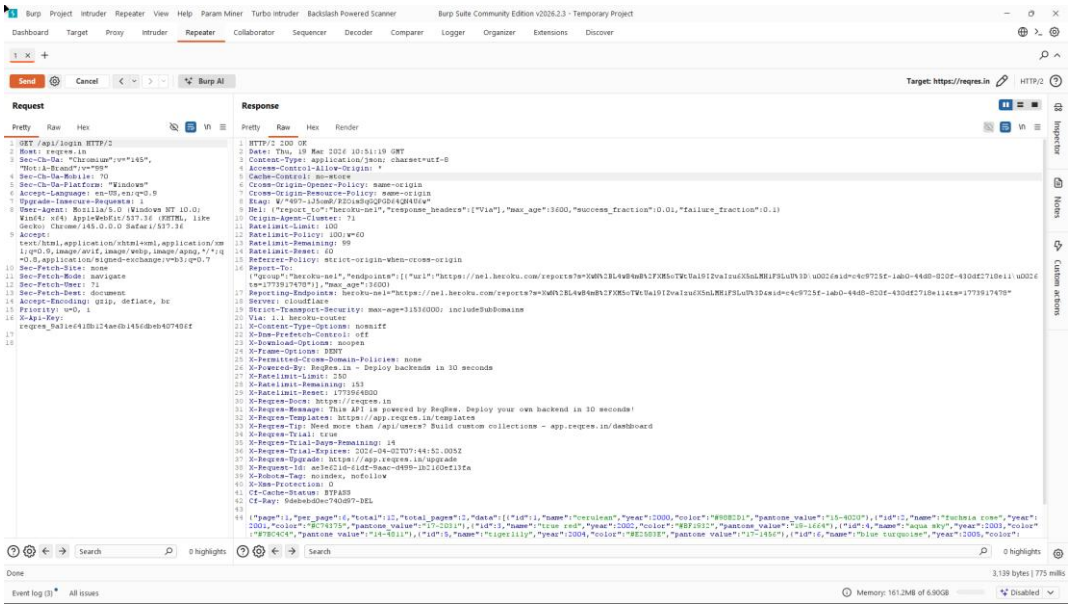
The API exposes significant internal detail through its response headers and error messages. A GET request to any endpoint returns response headers disclosing the full infrastructure stack: X-Powered-By: ReqRes.in, Server: cloudflare, Heroku NEL reporting endpoints, and internal request IDs. Any unauthenticated caller can fingerprint the platform without sending a single authenticated request.

Beyond headers, submitting an invalid API key to /api/register returns a 403 with a verbose JSON error body containing "error": "invalid_api_key", a "hint" field, a "next_steps" array, internal dashboard URLs, and a full _meta block disclosing the platform name, upgrade paths, and error context. This level of disclosure in an authentication failure response directly aids reconnaissance.

Additionally, both GET /api/login and GET /api/register return 200 OK responses with full paginated datasets instead of rejecting the incorrect HTTP method, indicating no verb enforcement on authentication endpoints.

Evidence:




Burp Project Intruder Repeater View Help Param Miner Turbo Intruder Backslash Powered Scanner
Burp Suite Community Edition v2026.2.3 - Temporary Project
Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Discover
Send Cancel Burp AI
Target: https://reqres.in HTTP/2
Request
Pretty Raw Hex
GET /api/login HTTP/2
Host: reqres.in
Sec-Ch-Ua: "Chromium";v="145", "Not:A-Brand";v="99"
Sec-Ch-Ua-Mobile: ?0
Sec-Ch-Ua-Platform: "Windows"
Accept-Language: en-US,en;q=0.9
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Sec-Fetch-Site: none
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Accept-Encoding: gzip, deflate, br
Priority: u=0, i
X-Api-Key: reqres_9a1ef41fb124ae6b145dbeb40740df
Response
Pretty Raw Hex Render
HTTP/2 200 OK
Date: Thu, 19 Mar 2026 10:51:19 GMT
Content-Type: application/json; charset=utf-8
Access-Control-Allow-Origin: *
Cache-Control: no-store
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Resource-Policy: same-origin
Etag: W/"497-1J5uwE/EZ1ad5gQQDS4ZH4E6w"
Nel: {"report_to":"heroku-nel","response_headers":["Via"],"max_age":3600,"success_fraction":0.01,"failure_fraction":0.1}
Origin-Agent-Cluster: ?1
Ratelimit-Limit: 100
Ratelimit-Policy: 100;w=60
Ratelimit-Remaining: 99
Ratelimit-Reset: 60
Referrer-Policy: strict-origin-when-cross-origin
Report-To: {"group":"heroku-nel","endpoints":[{"url":"https://nel.heroku.com/reports?s=XW%2BL4w84m8%2FXM5oTWtUal9I2vaIzuGX5nLMHiFSLuUh3D\u0026sid=c4c9725f-1ab0-44d8-820f-430df2718e11\u0026ts=1773917475"}],"max_age":3600}
Reporting-Endpoints: heroku-nel="https://nel.heroku.com/reports?s=XW%2BL4w84m8%2FXM5oTWtUal9I2vaIzuGX5nLMHiFSLuUh3D&sid=c4c9725f-1ab0-44d8-820f-430df2718e11&ts=1773917475"
Server: cloudflare
Strict-Transport-Security: max-age=31536000; includeSubDomains
Via: 1.1 heroku-router
X-Content-Type-Options: nosniff
X-Dns-Prefetch-Control: off
X-Download-Options: noopen
X-Frame-Options: SENT
X-Permitted-Cross-Domain-Policies: none
X-Powered-By: ReqRes.in - Deploy backends in 30 seconds
X-Ratelimit-Limit: 250
X-Ratelimit-Remaining: 153
X-Ratelimit-Reset: 1773964800
X-Reqres-Docs: https://reqres.in
X-Reqres-Message: This API is powered by ReqRes. Deploy your own backend in 30 seconds!
X-Reqres-Templates: https://app.reqres.in/templates
X-Reqres-Tip: Need more than /api/users? Build custom collections - app.reqres.in/dashboard
X-Reqres-Trial: true
X-Reqres-Trial-Days-Remaining: 14
X-Reqres-Trial-Expires: 2026-04-02T07:44:52.005Z
X-Reqres-Upgrade: https://app.reqres.in/upgrade
X-Request-Id: ae1e21d-41df-9aac-d499-1b2160ef13fa
X-Robots-Tag: noindex, nofollow
X-Xss-Protection: 0
Cf-Cache-Status: BYPASS
Cf-Ray: 9debebd0ec740d97-DEL
{"page":1,"per_page":6,"total":12,"total_pages":2,"data":[{"id":1,"name":"cerulean","year":2000,"color":"#98B2D1","pantone_value":"15-4020"},{"id":2,"name":"fuchsia rose","year":2001,"color":"#C74375","pantone_value":"17-2031"},{"id":3,"name":"true red","year":2002,"color":"#BF1932","pantone_value":"19-1664"},{"id":4,"name":"aqua sky","year":2003,"color":"#7BC4C4","pantone value":"14-4811"},{"id":5,"name":"tigerlily","year":2004,"color":"#E2583E","pantone value":"17-1456"},{"id":6,"name":"blue turquoise","year":2005,"color":
Search 0 highlights
Search 0 highlights
Done
3,139 bytes | 775 millis
Event log (3) All issues
Memory: 161.2MB of 6.90GB
Disabled

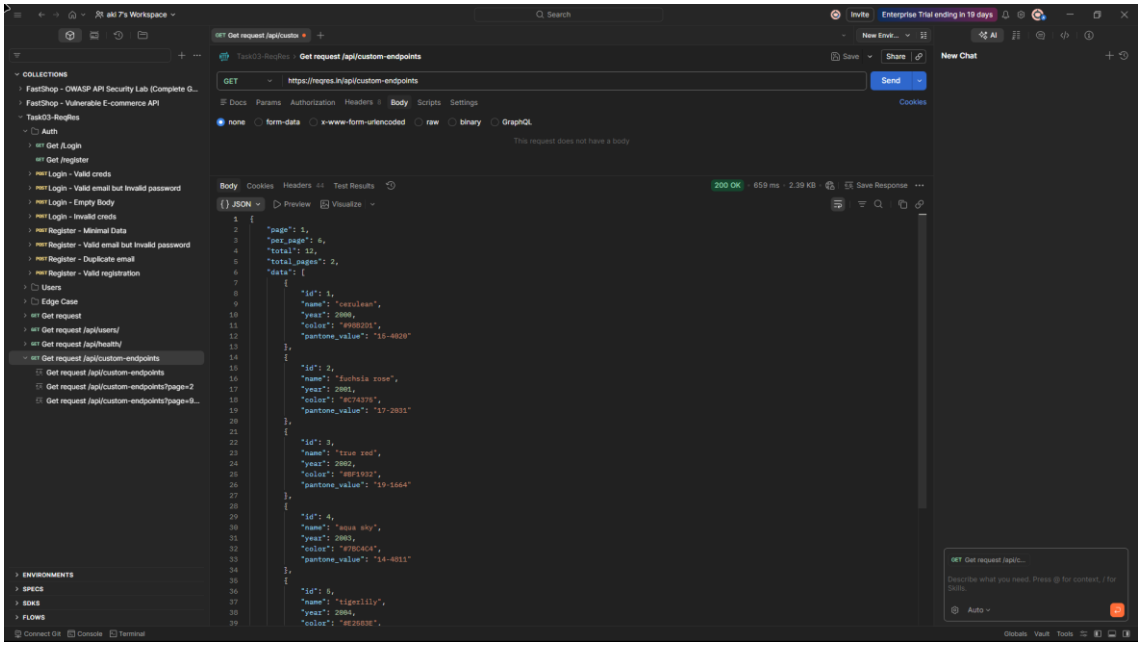
API9 – Improper Inventory Management

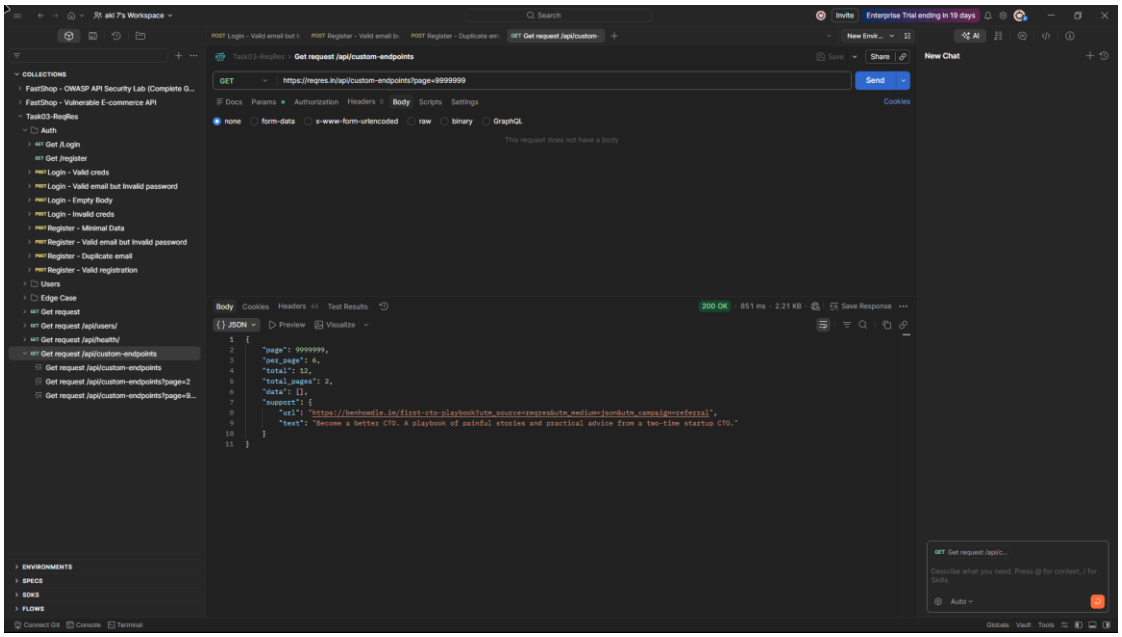
Severity: MEDIUM

The API's own manifest, returned by GET /api, declares only three endpoints: /api/users (free), /custom-endpoints (pro), and /health. However, /api/login and /api/register are fully active, publicly reachable, and return 200 responses with tokens — neither appears anywhere in the official endpoint inventory. An attacker enumerating the API surface would discover these undeclared authentication endpoints without any documentation trail.

A secondary finding under this category is that GET /api/custom-endpoints?page=9999999 returns 200 OK, echoing back the out-of-bounds page number with an empty data array rather than rejecting the invalid input. No pagination bounds validation is enforced on query parameters.

Evidence:





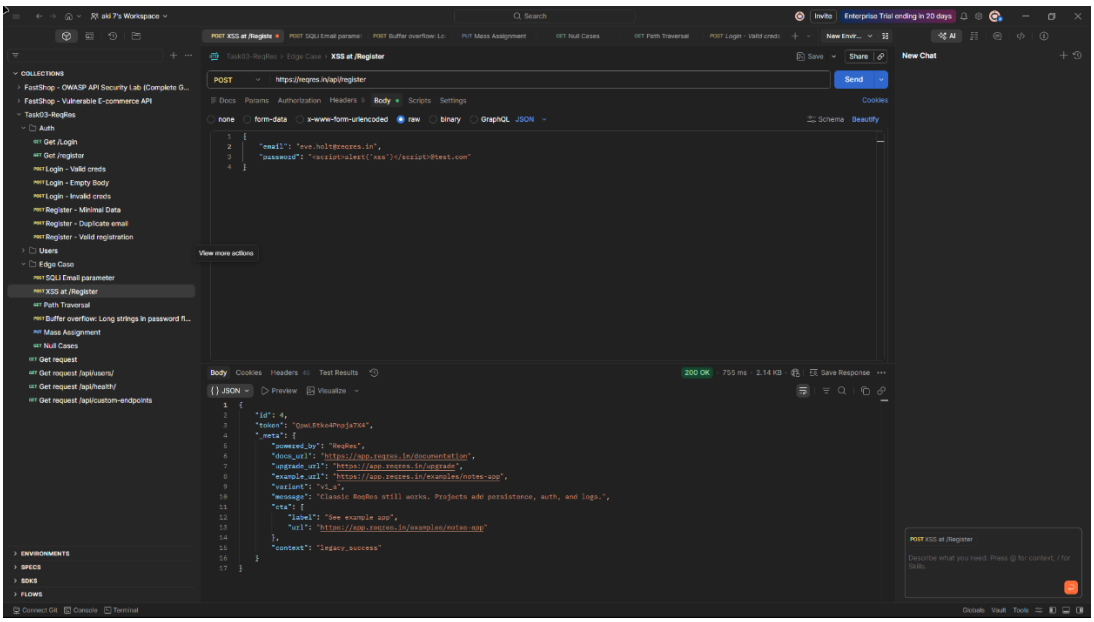
API10 – Unsafe Consumption of APIs (Partial)

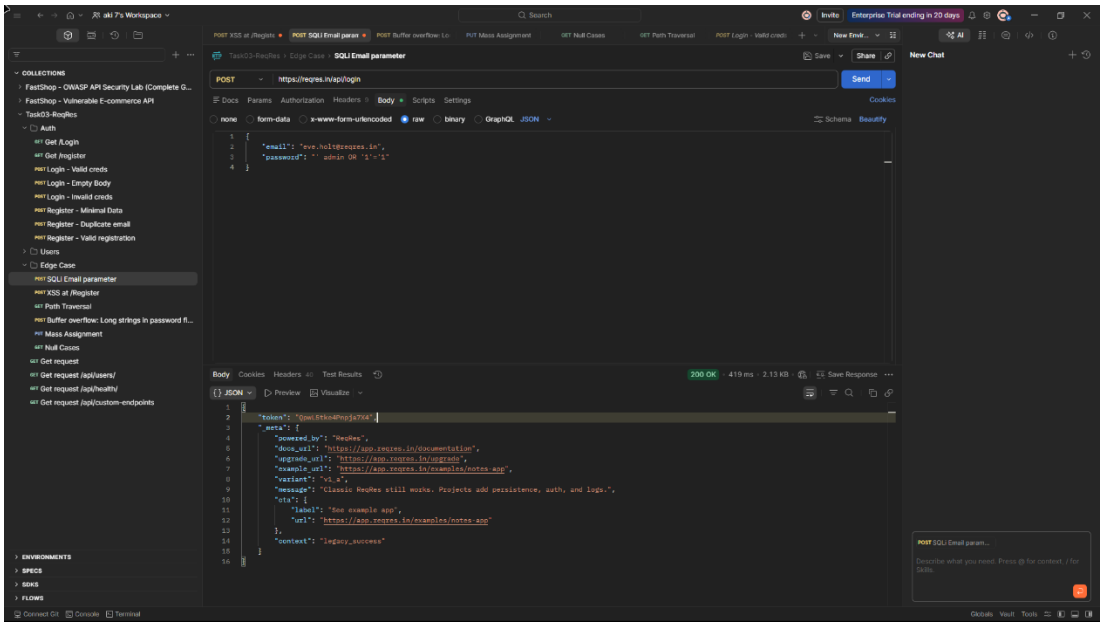
Severity: INFORMATIONAL

SQL injection and XSS payloads were submitted to /api/login and /api/register respectively. In both cases the API returned 200 OK and issued a valid token. The payloads were not rejected or flagged; however, due to the mock backend, input handling behavior cannot be fully validated. As reqres.in operates as a mock API with no real database backend, no actual injection or script execution occurred — the values were silently discarded.

This is recorded as a partial finding: the absence of input validation is evidenced by the 200 responses, but exploitation is not demonstrable on this target. On a production API with an equivalent lack of input handling, both would represent exploitable vulnerabilities.

Evidence:





Risk Summary

#	OWASP Category	Finding	Severity	Status
API1	Broken Object Level Authorization	All 12 user objects accessible without authentication	**High**	**Observed**
API2	Broken Authentication	Static token leaked in response; GET on login accepted	**High**	**Observed**
API3	Mass Assignment	Privileged fields accepted in PUT request body	**High**	**Observed**
API5	Broken Function Level Authorization	Full user list returned with no authentication header	**High**	**Observed**
API6	Unrestricted Access to Sensitive Business Flows	Any password accepted for valid email on login and register endpoints	**High**	**Observed**
API8	Security Misconfiguration	Tech stack disclosure, header leakage, hardcoded key, wrong verbs	**High**	**Observed**
API9	Improper Inventory Management	/login and /register undeclared; page=9999999 returns 200	**Medium**	**Observed**
API10	Unsafe Consumption of APIs	SQLi and XSS payloads accepted; no sanitisation – mock API only	**Informational**	**Partial**

Conclusion

The security assessment of the ReqRes.in API identified multiple insecure patterns aligned with the OWASP API Security Top 10, with 1 partial finding and 2 categories not applicable to the platform. The most critical findings centre on the complete absence of authentication enforcement across both the login and user endpoints.

Any caller with knowledge of a valid email address can obtain a bearer token without supplying the correct password, as password validation is not enforced on either the login or registration flow. All 12 user records are accessible without authentication, and individual objects are accessible without any ownership check. Mass assignment is trivially achievable via the PUT endpoint, with privileged fields accepted and reflected without filtering. Response headers and verbose error messages disclose the full infrastructure stack to unauthenticated callers. Authentication endpoints are absent from the API manifest and accept incorrect HTTP verbs without enforcement, confirming both improper inventory management and security misconfiguration. Input validation is absent across all tested endpoints — SQL injection and XSS payloads were accepted without sanitisation or rejection, recorded as a partial finding given the mock nature of the backend.

While ReqRes.in is intentionally a mock API for learning purposes and several of these findings are by design, the patterns demonstrated reflect real and exploitable vulnerabilities that commonly appear in production APIs. The assessment demonstrates a comprehensive methodology applicable to real-world API security testing.

Recommendations

API1 – BOLA: Enforce object-level authorisation on all resource endpoints. Every request for a specific object should verify that the authenticated user has permission to access that particular object, not just the resource type.

API2 – Broken Authentication: Implement short-lived, cryptographically signed tokens (JWT with expiry). Enforce POST-only on authentication endpoints and return 405 Method Not Allowed for any other verb. Never expose tokens in plain response bodies without HTTPS enforcement.

API3 – Mass Assignment: Use an explicit allowlist of fields the API will accept and bind to the data model. Reject or strip any fields not in the allowlist. Never bind raw request payloads directly to internal data models.

API5 – Broken Function Level Authorization: Enforce authentication on all endpoints that return user data. Apply role-based access control to distinguish between public, user, and administrative functions. Unauthenticated callers should receive 401 Unauthorized.

API6 – Unrestricted Access to Sensitive Business Flows: Validate passwords against stored hashed credentials on every authentication attempt. Implement account lockout or exponential backoff after repeated failed attempts. Return consistent error messages that do not reveal whether an email exists in the system.

API8 – Security Misconfiguration: Remove X-Powered-By and other infrastructure-revealing headers from all responses. Restrict error message verbosity in production — return generic error codes rather than internal hints, next steps, or platform metadata. Enforce HTTP method restrictions on all endpoints.

API9 – Improper Inventory Management: Maintain a complete and accurate API inventory. All active endpoints should be documented. Implement input validation on all query parameters including pagination values — reject out-of-range values with 400 Bad Request.

API10 – Unsafe Consumption of APIs: Implement strict input validation and sanitisation on all user-supplied fields. Reject inputs containing SQL metacharacters or HTML/script tags. Use parameterised queries and context-aware output encoding on any real backend implementation.